

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

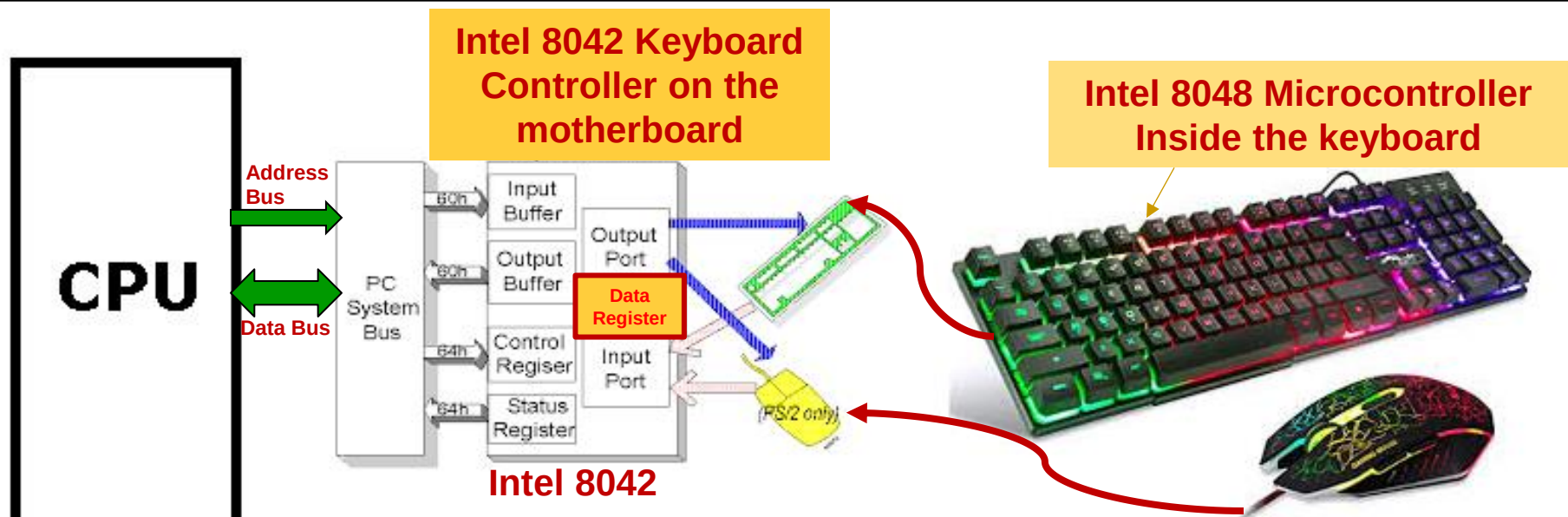
OPERATING SYSTEM

WEEK 7 LECTURE 2

WRITING LINUX KERNEL MODULE FOR KEYBOARD DEVICE

Write a Kernel module to display the scan code of the key being pressed and released using keyboard interrupt routine

WRITING A KEYBOARD INT KERNEL MODULE



Keyboard controller or Intel 8042 keyboard controller is found on the motherboard. The controller handles input received from the computer keyboard, A20 lines, reset, deciphering scan codes, as well as the PS/2 mouse.

Keyboard Controller Data Register port address is 0x60
Keyboard Controller Status Register port address is 0x64

WRITING A KEYBOARD INT KERNEL MODULE

KEYBOARD BASICS:

Keyboard Controller used is Intel 8042

IRQ number associated with Keyboard is 1

```
#define KBD_IRQ          1          /* IRQ number for keyboard (i8042) */
```

Keyboard Controller Data Register port address is 0x60. Whenever a key is pressed or released, the scan code of the key being pressed or released is stored in Data Register

```
#define KBD_DATA_REG      0x60      /* I/O port for keyboard data */
```

Most significant bit of the scan code indicates whether key is pressed (Bit 7=0) or key is released (Bit 7=1).

Remaining lower significant bits (Bit6-0) of the scan code indicates the type of key (Numeric, alphabets, control) being pressed or released.

```
#define KBD_SCANCODE_MASK 0x7f /* to retrieve scan code */
```

```
#define KBD_STATUS_MASK   0x80 /* to know key pressed or released */
```

Use function `inb(port address)` to Read a byte from port `KBD_DATA_REG`

```
scancode = inb(KBD_DATA_REG);
```

```
scancode & KBD_SCANCODE_MASK provides scancode of the key
```

```
scancode & KBD_STATUS_MASK indicates "Key Released" if true and "Key Pressed" if false
```

Instead of using `printk(KERNEL_INFO format string, args)` use the equivalent kernel function `pr_info ( format string, args)`

Write a keyboard ISR to read the scancode when a key is pressed or released on the keyboard

```
static irqreturn_t kbd2_isr(int irq, void *dev_id)
```

Write `__init kbd` function to invoke irq request

```
request_irq(KBD_IRQ, kbd2_isr, IRQF_SHARED, "kbd2", (void *)kbd2_isr);
```

Write `__exit kbd` function to free irq

```
free_irq(KBD_IRQ, (void *)kbd2_isr);
```

```
$ mkdir kbdInt && cd kbdInt && gedit kbdInt.c
```

WRITING A KEYBOARD INT KERNEL MODULE

```
#include <linux/kernel.h>                                /* kbdInt.c */
#include <linux/module.h>
#include <linux/interrupt.h>
#include <asm/io.h>

#define KBD_IRQ          1          /* IRQ number for keyboard (i8042) */
#define KBD_DATA_REG     0x60       /* I/O port for keyboard data */
#define KBD_SCANCODE_MASK 0x7f
#define KBD_STATUS_MASK  0x80

static irqreturn_t kbd2_isr(int irq, void *dev_id)
{
    char scancode;
    scancode = inb(KBD_DATA_REG);
    pr_info("Scan Code %x %s\n", scancode & KBD_SCANCODE_MASK, scancode & KBD_STATUS_MASK ? "Released"
        : "Pressed");
    return IRQ_HANDLED;
}

static int __init kbd2_init(void)
{
    return request_irq(KBD_IRQ, kbd2_isr, IRQF_SHARED, "kbd2", (void *)kbd2_isr);
}

static void __exit kbd2_exit(void)
{
    free_irq(KBD_IRQ, (void *)kbd2_isr);
}

module_init(kbd2_init);
module_exit(kbd2_exit);
MODULE_LICENSE("GPL");
```

WRITING A KEYBOARD INT KERNEL MODULE

\$ vi Makefile

obj-m +=kbdInt.o

all:

**make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD)
modules**

clean:

**make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD)
clean**

\$make

- ★ **Open another terminal by using xterm and in that terminal type**

\$tail -f /var/log/kern.log

\$sudo insmod kbdInt.ko

Write a Kernel module to identify a key being pressed or released on the keyboard and display the scan code and name of the key being pressed and released using keyboard interrupt routine.

WRITING A KEYBOARD INT KERNEL MODULE

```
#include <linux/kernel.h>                                /* FILE NAME : kbdAction.c
#include <linux/module.h>
#include <linux/interrupt.h>
#include <asm/io.h>
#define KBD_IRQ 1                                         /* IRQ number for keyboard (i8042) */
#define KBD_DATA_REG 0x60                               /* I/O port for keyboard data register*/
#define KBD_STATUS 0x64                                 /* I/O port for keyboard status register */
#define KBD_SCANCODE_MASK 0x7f
#define KBD_STATUS_MASK 0x80
char key_name[] = " 1234567890-= qwertyuiop[] asdfghjkl;' zxcvbnm,./";
static irqreturn_t kbdInt_isr(int irq, void *dev_id)
{
    char scancode;char keypress;
    int status;
    status = inb(KBD_STATUS);
    pr_info("Your keyboard status is %d\n",status);
    scancode = inb(KBD_DATA_REG);
    keypress = key_name[scancode & KBD_SCANCODE_MASK];
    if(!(scancode & KBD_STATUS_MASK))
        pr_info("You have pressed the Key : %c , its Scan Code is %x and ASCII code - %d\n", keypress,
            scancode & KBD_SCANCODE_MASK, keypress);
    if(scancode & KBD_STATUS_MASK)
        pr_info("Now you have released the key %c and its Scan Code is %x\n", keypress, scancode);
    return IRQ_HANDLED;
}
```

WRITING A KEYBOARD INT KERNEL MODULE

/* FILE NAME : kbdAction.c

```
static int __init kbdInt_init(void)
{
    return request_irq(KBD_IRQ, kbdInt_isr, IRQF_SHARED, "kbdInt", (void *)kbdInt_isr);
}
```

```
static void __exit kbdInt_exit(void)
{
    free_irq(KBD_IRQ, (void *)kbdInt_isr);
}
```

```
module_init(kbdInt_init);
module_exit(kbdInt_exit);
MODULE_LICENSE("GPL");
```

\$ vi Makefile

obj-m +=kbdAction.o

all:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) modules

clean:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) clean

\$make

* Open another terminal by using xterm and in that terminal type

\$tail -f /var/log/kern.log

\$sudo insmod kbdAction.ko

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 4 & APPENDIX B

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

INTERRUPTS & EXCEPTIONS

❑ Questions

- * With the help of a block diagram, show how the Keyboard is interfaced to the Processor. Also describe the operations performed.
- * `pr_info(format, args)` is equivalent kernel function for
- * Keyboard Controller used in IBM Compatible PC is
 - a) Intel 8042 b) Intel 8048 c) Intel 8086 d) Intel 8259
- * Microcontroller used inside the keyboard to scan the keys to detect whether any key is being pressed/released and to generate the scan code corresponding to the key being pressed/released
 - a) Intel 8042 b) Intel 8048 c) Intel 8086 d) Intel 8259
- * Intel 8042 Keyboard Controller is found on the
 - a) Motherboard b) Inside the keyboard c) Inside the CPU d) Inside the PIC
- * Intel 8048 Microcontroller is found on the
 - a) Motherboard b) Inside the keyboard c) Inside the CPU d) Inside the PIC
- * IRQ number associated with keyboard is
 - a) 0 b) 1 c) 6 d) 13
- * Keyboard Controller Data Register port address is
 - a) 0x60 b) 0x64 c) 0x7F d) 0x80
- * Keyboard Controller Status Register port address is
 - a) 0x60 b) 0x64 c) 0x7F d) 0x80
- * The C function used to read a byte from either data register or status register of a keyboard controller by specifying its corresponding port address is
 - a) `inb(port_address)` b) `inputb(port_address)`
 - c) `outb(port_address)` d) `outputb(port_address)`

SOC 3010

OPERATING SYSTEM

END OF
WEEK 7 LECTURE 2

WRITING LINUX KERNEL
MODULE FOR
KEYBOARD DEVICE

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

